Evaluating Feedback in LLM-Guided Heuristic Evolution
for Minimum Weighted Dominating Set

Group Composition:
Nguyen Minh Tung, A0343185A

Github Repository: `https://github.com/junghyukie/Final_Assignment`

Abstract

Large language models can discover optimization heuristics through iterative program generation and evaluation. Whether parent conditioning and numerical performance feedback improve the resulting heuristics beyond simpler generation strategies remains unclear, particularly when gains during search may not generalize. This paper investigates that question on the minimum weighted dominating set problem using COVER-EVO, a controlled study in which the language model proposes vertex-scoring expressions within a fixed construction and pruning scaffold. Independent generation, parent-conditioned evolution, and evolution with numerical feedback are compared with parameter search, handcrafted priorities, and classical optimization. Evaluation combines controlled synthetic graph distributions with transfer of frozen heuristics to a published benchmark. Under the tested conditions, neither the combined evolutionary strategy nor the incremental prompt-feedback policy establishes a consistent held-out advantage. Handcrafted priorities remain competitive across the tested distributions, while exact optimization is the stronger control on the original small graphs. These findings support evaluating LLM-guided heuristic evolution through the quality and generalization of frozen programs rather than improvements observed during search.

1 Introduction

FunSearch [1], Evolution of Heuristics (EoH) [2], and ReEvo [3] combine language-model program generation with performance-guided search. Their success raises a narrower question: do parent conditioning and numerical performance feedback improve the final heuristics beyond independent generation? Improvements during search can reflect adaptation to the training instances rather than better decisions on unseen problems.

I study this question on the minimum weighted dominating set (MWDS) problem, where weighted greedy provides an interpretable reference and exact optimization certifies quality on small graphs. COVER-EVO holds construction, pruning and feasibility checking fixed while the LLM proposes vertex-scoring expressions. Independent generation (IND), parent-only evolution (EVO-P), and evolution with numerical prompt feedback (EVO-F) are compared with parameter search and handcrafted controls. Each branch develops its own archive and parent history, so the comparisons concern complete proposal policies.

The evaluation addresses three questions. **RQ1:** Do the evolutionary proposal policies improve held-out quality over independent generation? **RQ2:** How do the frozen heuristics transfer across graph distributions, and what do feature interventions reveal about their behavior? **RQ3:** How do solution quality and execution efficiency compare with non-LLM controls? The contribution is a controlled comparison of proposal policies, an evaluation of frozen-program transfer, and an analysis of how scoring changes survive construction and pruning. Synthetic shifts separate cost allocation from topology, while a published MWDS benchmark tests external transfer.

2 Background and MWDS

2.1 Weighted Domination as Set Cover

Let $G = (V, E)$ be an undirected graph with $n = |V|$ vertices and positive integer costs c_v. The closed neighborhood is $N[v] = \{v\} \cup \{u : (u, v) \in E\}$. MWDS seeks a subset $D \subseteq V$ minimizing $C(D) = \sum_{v \in D} c_v$ such that $\bigcup_{v \in D} N[v] = V$. An equivalent integer formulation is

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & \sum_{v \in V} c_v x_v, && (1) \\ \text{s.t.} \quad & \sum_{v \in N[u]} x_v \geq 1, && \forall u \in V, \\ & x_v \in \{0, 1\}, && \forall v \in V. \end{aligned}$$

Each closed neighborhood is a set with selection cost c_v, making MWDS an instance of weighted set cover. Given uncovered vertices U, standard weighted greedy selects a vertex maximizing $|N[v] \cap U|/c_v$. It achieves an approximation ratio of $H_d \leq 1 + \ln d$ for $d = \max_{v \in V} |N[v]|$ [4]. Removing redundant selected vertices while preserving coverage retains this guarantee. Arbitrary learned scoring functions have no such guarantee.

2.2 Related Work and Design Rationale

AlphaEvolve and LLaMEA [5, 6] allow more flexible algorithmic generation. Restricting COVER-EVO to scoring expressions keeps construction and feasibility

fixed across proposal strategies, but excludes changes to the broader solver. This is a design trade-off within heuristic generation [7]. Classical methods such as FastMWDS [8] combine reductions and local search for large-scale weighted domination; the objective is total cost rather than cardinality [9]. Reported generalization failures for LLM-evolved bin-packing heuristics [10] motivate testing frozen programs beyond their training distribution. A handcrafted rarity-weighted coverage priority, RGR+P, provides an interpretable control within the same scaffold. Section 6.2 develops its mathematical interpretation.

3 COVER-EVO Method

3.1 Fixed Construction and Pruning

COVER-EVO uses a fixed greedy construction procedure in which only the vertex-scoring function is evolved. Starting with an empty dominating set $D = \varnothing$ and uncovered vertices $U = V$, the algorithm repeatedly selects the unselected vertex v with the highest score $f(v, U)$ among those satisfying $|N[v] \cap U| > 0$. The selected vertex is added to D, and its closed neighborhood is removed from U. Ties are resolved by lower vertex cost, then lower vertex ID. Previously covered vertices remain eligible if they can cover additional uncovered vertices.

Once all vertices are covered, a deterministic pruning pass examines selected vertices in descending cost order, breaking ties by ascending ID. A vertex is removed only if the remaining set still dominates the graph. This construction and pruning procedure is shared by all scoring-based methods.

Feasibility and objective values are independently verified against the original graph. Operational failures receive a penalty equal to the cost of selecting all vertices and remain included in evaluation statistics.

3.2 Scoring Features and DSL

For a candidate vertex v, define its newly covered vertices as $A_v = N[v] \cap U$, coverage gain $k_v = |A_v|$, and mean vertex cost $\bar{c} = n^{-1} \sum_{v \in V} c_v$. The scoring language exposes five features:

$$\begin{aligned} g_v &= \frac{k_v}{|U|}, & q_v &= \frac{c_v}{\bar{c}}, \\ r_v &= \frac{1}{k_v} \sum_{u \in A_v} \frac{1}{|N[u]|}, & o_v &= 1 - \frac{k_v}{|N[v]|}, \\ t &= \frac{|U|}{n}. \end{aligned} \tag{2}$$

Here, g_v measures normalized coverage gain, q_v normalized selection cost, and r_v the average rarity of newly covered vertices. The overlap feature o_v measures the fraction of the candidate's neighborhood already covered, while t represents the remaining uncovered fraction. Rarity is computed using original neighborhood sizes rather than dynamically updated degrees.

Programs are scalar expressions over these features and bounded constants, using arithmetic, protected division, minimum, maximum and absolute value. Control flow, assignments, imports and external calls are forbidden. The repository specifies expression-size limits and numerical safeguards.

The initial greedy seed is $f_0(v, U) = g_v/q_v$, which induces the same ranking as k_v/c_v. I additionally consider a handcrafted rarity-weighted rule:

$$f_{\mathrm{RGR}}(v, U) = \frac{g_v r_v}{q_v}. \tag{3}$$

GR+P denotes standard weighted greedy with pruning, while RGR+P uses Eq. (3) with the same construction and pruning procedure. RGR+P is a rarity-weighted coverage priority; Section 6.2 derives its equivalence to an inverse-neighborhood-weighted demand sum per unit cost. Candidate priorities are evaluated using exact rational arithmetic. Reverse-delete provides an additional baseline by starting from all vertices and applying the pruning procedure directly.

3.3 LLM-Guided Discovery

Each search run begins with the greedy seed and evaluates candidate programs on the training graphs using mean cost ratio to GR+P. An archive maintains the four best candidates, ranked by training fitness, AST size, and program hash. Every proposal attempt counts toward the budget without replacement.

IND generates candidates independently, without incorporating previous programs or evaluation results into the generation prompt. It nevertheless uses the same evaluator, archive, and final selection procedure as the evolutionary conditions.

EVO-P conditions new proposals on up to two sampled archive parents, providing their expressions and rationales without explicit numerical performance feedback.

EVO-F follows the same parent-conditioned procedure but additionally provides numerical training summaries and descriptors of poorly performing cases. Both evolutionary conditions begin with eight fresh proposals and subsequently include one fresh proposal after every four parent-conditioned attempts.

EVO-P still uses training fitness for archive selection. EVO-F versus EVO-P therefore compares a prompt-feedback policy with a parent-only policy, both using fitness-based selection. It adds numerical summaries and case descriptors, while subsequent archives and parent exposure can diverge. The contrast concerns complete proposal policies. EVO-F versus IND compares the combined parent-conditioned and feedback-guided policy with independent generation.

3.4 Controls and Winner Selection

To assess whether simple parameter variation can produce competitive heuristics, I include RS-PARAM, a random search over the scoring family

$$f_{a,b,d}(v, U) = \frac{g_v}{q_v}(1 + a r_v + b o_v + d r_v t). \tag{4}$$

Coefficients a, b, d are independently sampled from $\{-2.000, -1.999, \ldots, 2.000\}$. RS-PARAM uses the same proposal-attempt budget, training evaluator, greedy seed, and validation procedure, without LLM calls. Its parameterized search space is not identical to the full expression language.

Following discovery, the three highest-ranked training candidates from each run are evaluated on the validation graphs. One winner is selected per run and frozen before held-out testing. Validation results are never supplied to the proposal model.

4 Experimental Protocol

4.1 Graphs, Costs, and Shifts

The geometric generator samples uniform positions in the unit square and retains the $3(n-3)$ closest pairs. This matches the edge count of the NetworkX Barabási–Albert (BA) construction with $m = 3$ [11], not its other structural properties. Labels are randomized and disconnected graphs retained. Training and validation contain 30 and 20 distinct geometric graphs at $n = 40$; their topologies are disjoint from the test set.

Costs are integers from 1 to 10. Each of the 60 primary in-distribution (ID) topologies at $n = 40$ receives random, hub-expensive and hub-cheap assignments of the same cost multiset. The latter two align cost positively or negatively with degree. Size and topology shifts add 60 geometric graphs at $n = 80$ and 60 BA graphs at $n = 40$, both with random costs. The test therefore contains 300 weighted instances from 180 distinct topologies; the three ID cost variants are dependent.

4.2 Model and Discovery Budget

Gemini 3.6 Flash uses temperature 0.8, a 2,048-token response limit and low reasoning effort. Each condition has five independent runs of 40 attempts with fixed settings; validation winners are frozen before testing. Matched attempts do not imply matched numbers of unique executable candidates.

4.3 Metrics and Uncertainty

For method h, discovery run r and instance i, excess cost is

$$e_{hri} = 100 \left(\frac{C_{hri}}{C_{\mathrm{GR+P},i}} - 1 \right). \tag{5}$$

I average individual ratios; negative excess favors h. EVO-F minus IND on ID/random is the prespecified primary contrast; EVO-F minus EVO-P and EVO-P minus IND are secondary. The original crossed bootstrap resamples run and graph blocks while pairing run indices across methods. Equal indices need not match search runs, so the sensitivity resamples method runs independently with paired graphs. Both percentile analyses use 5,000 draws and seed 4212, conditional on one training corpus [12].

CP-SAT solves Eq. (1) with one worker and a 20-second search limit. Only OPTIMAL status certifies C_i^* [13]; feasible incumbents are not exact references. All 300 original instances are certified. Optimum gap is $100(C_{hri}/C_i^* - 1)$, and an optimal hit requires $C_{hri} = C_i^*$. Feasibility and cost are independently checked. Original runtime summaries are medians of pair-level medians from a warmup and three uncached repeats in randomized order; fixed-baseline outcome counts use unique graphs.

The same-session primary control gives CP-SAT50 a 50 ms end-to-end target, including hint construction, setup, extraction and checking; search gets the remaining time. GR+P, RGR+P and five EVO-F winners use the same warmup, randomized order and three repeats. A feasible greedy fallback is available, and overruns remain included.

4.4 Frozen-Program Diagnostics

Post-hoc feature interventions zero rarity or overlap, or omit pruning, without retraining or reselection. All retain the original pruned-greedy denominator. Ten seeded relabelings per primary graph move costs with vertices and recompute greedy; these are dependent perturbations, not additional independent graphs. Behavioral comparisons use saved construction sequences on original vertex labels, pre- and post-pruning membership sets, and final costs separately. Non-seed summaries subset the existing intervention records; all diagnostics remain descriptive.

4.5 Published External Benchmark

Post-hoc transfer uses the published T1/T2 instances of Jovanović, Tuba and Simian [14]. Selection is fixed before evaluation: $n \in \{100, 300, 500\}$, edge budgets n and $10n$, and seeds 0–3. The 48 weighted instances share 24 topologies, with random weights in T1 and degree-dependent weights in T2. All 20 winners and three handcrafted baselines run without tuning; original labels and weights are retained. Family means use all selected instances. External runtimes are descriptive first-call medians; identical programs share measurements. Full outcome counts and native-format inputs are available in the repository.

5 Results

5.1 Primary Held-Out Performance

EVO-F has the lowest mean excess among LLM strategies, but no discovered method outperforms GR+P on average (Table 1). The primary EVO-F–IND contrast is -0.148 percentage points (original run-index-paired 95% CI: $[-0.699, 0.220]$). The secondary EVO-F–EVO-P contrast is -0.305 points ($[-1.004, 0.174]$); EVO-P–IND is $+0.157$ ($[-0.659, 1.026]$). These intervals establish neither a reliable advantage nor equivalence.

Table 1: Primary test on 60 random-cost geometric graphs ($n = 40$). Excess is relative to GR+P. Search-method counts cover $5 \times 60 = 300$ run–graph pairs; fixed baselines use 60 unique graphs. Runtime includes pruning. Lower excess and optimum gap are better.

Method	Excess (%)	Optimum gap (%)	Optimal hits (%)	Wins/ties/losses	Median (ms)
IND	+0.462	3.359	61.3	4/270/26	23.40
EVO-P	+0.618	3.520	59.3	6/257/37	21.82
EVO-F	+0.313	3.209	63.3	4/277/19	17.26
RS-PARAM	+0.614	3.473	61.7	28/224/48	39.56
GR+P	+0.000	2.889	66.7	0/60/0	4.23
RGR+P	+0.482	3.372	63.3	4/49/7	10.83
Reverse-delete	+2.367	5.235	46.7	6/37/17	0.54

Independent method-run resampling gives intervals of $[-0.890, 0.512]$ for EVO-F–IND and $[-1.049, 0.314]$ for EVO-F–EVO-P (Figure 1), leaving the interpretation unchanged. Greedy is already optimal on 40 of 60 graphs. On the remaining 20, EVO-F closes no mean headroom (-0.25% of the available gap). Limited headroom therefore constrains, but does not fully explain, the lack of improvement.

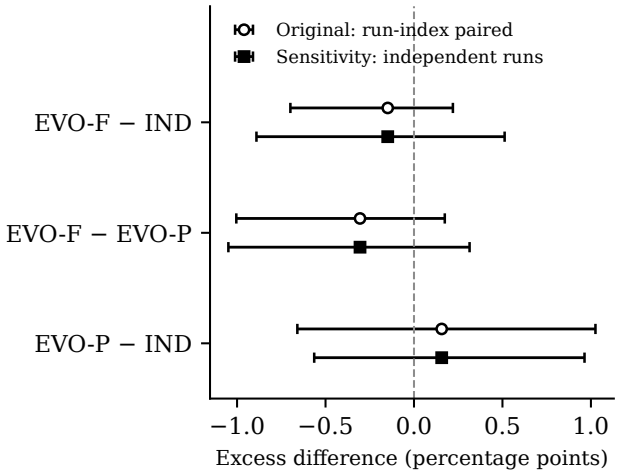


Figure 1: Primary and secondary contrasts under the original run-index-paired bootstrap and the independent-run sensitivity. Both use paired graph indices, five runs per strategy and 5,000 draws. Negative values favor the first method; all intervals contain zero.

5.2 Synthetic Distribution Shifts

Controlled cost–degree alignment changes performance (Table 2). All LLM strategies improve on greedy on average with hub-expensive costs, yet the handcrafted RGR+P rule improves more. Because topology and the cost multiset are fixed, this comparison concerns cost allocation rather than cost scale.

On size and topology shifts, the LLM strategies lose to greedy on average. RGR+P improves in both, while RS-PARAM improves only on BA. A useful priority exists within the scaffold but is not consistently recovered by evolution. Section 6.2 analyzes its rarity weighting.

Table 2: Mean excess cost (%) on synthetic shifts, each with 60 graphs. Hub-cost variants share the primary topologies and cost multisets. Size80 changes graph size; BA changes topology. Primary ID/random values are in Table 1. Negative values favor the method.

Method	Hub-exp.	Hub-cheap	Size80	BA
IND	-0.378	+0.302	+0.213	+0.065
EVO-P	-0.579	+0.548	+0.294	+1.322
EVO-F	-0.615	+0.093	+0.168	+1.438
RS-PARAM	+0.827	+0.129	+0.401	-0.349
GR+P	+0.000	+0.000	+0.000	+0.000
RGR+P	-1.017	+0.088	-0.472	-0.483
Reverse-delete	+0.722	+1.547	+1.150	+13.048

5.3 Published T1/T2 Transfer

Table 3: External transfer: mean excess (%) and median first-call runtime (seconds, T1/T2). Each family has 24 weighted instances on shared topologies; search methods use all five frozen winners. Timing describes the reference interpreter. Full outcome counts and medians remain in the repository.

Method	T1 excess	T2 excess	Time T1/T2 (s)
GR+P	+0.00	+0.00	0.992/1.039
RGR+P	-1.26	-0.58	3.756/1.768
Reverse-delete	+6.55	+1.82	0.013/0.008
IND	+0.18	-0.08	4.011/2.881
EVO-P	+0.20	-0.03	4.104/2.954
EVO-F	+0.40	+0.16	3.842/2.628
RS-PARAM	+1.60	+0.80	8.377/5.257

Frozen programs transfer without benchmark-specific tuning. EVO-F has higher mean excess than IND and EVO-P in both weight families, while RGR+P has the lowest (Table 3). IND and EVO-P improve slightly on greedy under degree-dependent T2 weights; EVO-F does not. External family averages therefore show no descriptive feedback advantage. Size/density rankings are mixed.

RGR+P's gains are uneven: its wins/ties/losses against greedy are 11/11/2 on T1 and 7/15/2 on T2.

All external solutions pass independent feasibility checks. CP-SAT certifies only five T2 instances, all at $n = 100$, and no T1 instance within its search limit; exact-gap statistics on those five cases are not representative of either full family.

5.4 Classical Solver at 50 ms

Table 4: Same-session primary-graph control under a 50 ms end-to-end target. Fixed methods have 180 measured calls and EVO-F 900. Late counts calls exceeding the target; gaps and hits use certified optima.

Method	Gap (%)	Hits (%)	Median ms	Late
GR+P	2.89	66.7	4.17	0/180
RGR+P	3.37	63.3	10.50	0/180
EVO-F	3.21	63.3	14.94	0/900
CP-SAT50	0.00	100.0	7.39	0/180

CP-SAT50 reaches every primary optimum in every repeat, with median latency 7.39 ms versus 14.94 ms for EVO-F and no deadline overrun (Table 4). On these original $n = 40$ graphs, it offers both better solution quality and lower median latency for the evaluated reference implementation. This comparison concerns neither every possible optimized learned heuristic nor the external benchmark's longer certification attempts.

Discovery used 800 attempts, yielding 513 novel-valid proposals within runs and 287 invalid or duplicate proposals. The 600 LLM attempts cost US$0.0806 (IND), US$0.1115 (EVO-P), and US$0.1834 (EVO-F), including unsuccessful and duplicate proposals. Recorded training solver time was 58.2, 145.9, and 105.2 seconds, respectively; RS-PARAM used 323.6 seconds and no API calls. These totals include seed evaluations but exclude reference setup, parsing, search overhead, and API waiting; full evaluator wall time was not recorded. Discovery resources are separate from the deployment runtimes above, which measure execution of frozen heuristics.

6 Analysis of Selected Heuristics

6.1 Why Do Final Solutions Converge?

Across the five frozen EVO-F winners on the same 60 primary graphs, construction orders differ from GR+P in 161 of 300 run–graph observations. Only 61 have different pre-pruning vertex sets, 44 have different final sets, and 23 have different final costs. Orders are compared as complete sequences on the original labels; membership is compared as sets. These are descriptive frequencies over dependent observations, not 300 independent graph samples.

The stages discard different kinds of variation. In 100 observations, construction order changes without changing the constructed set. Of the 61 different constructed sets, 17 converge to the same final set after deterministic pruning. Another 21 observations retain different final sets with equal cost. Thus, trajectory diversity exceeds final-solution diversity, which in turn exceeds objective diversity. Pruning explains one part of this convergence; selection during construction and equal-cost alternatives explain the rest. A different scoring expression can therefore produce many different choices without improving the measured objective.

6.2 Why Does Rarity's Form Matter?

Seven of the 15 LLM winners are the greedy seed. Within EVO-F, runs 0 and 3 select that seed, while the other three select

$$\begin{aligned} f_1 &= (g + 0.4r)/q, \\ f_2 &= (g + 0.55r)/q, \\ f_4 &= g/q + 0.5r/q - 0.2o. \end{aligned} \tag{6}$$

The selected non-seed EVO-F programs therefore belong to a narrow additive-rarity family, with an overlap penalty in run 4. This characterizes the selected programs, not the full range of forms the model could generate.

For the additive component, the bonus relative to coverage gain is

$$f(v, U) = \frac{g_v + \alpha r_v}{q_v}, \qquad \frac{\alpha r_v}{g_v} = \frac{\alpha r_v |U|}{k_v}. \tag{7}$$

Coverage gain is normalized by the remaining demand $|U|$, whereas rarity averages inverse neighborhood sizes over the k_v newly covered vertices. The relative bonus can therefore grow with $|U|$ or decrease with k_v, even at the same average rarity. During construction, both the uncovered set and the composition of each candidate's newly covered neighborhood change. Graph size alone does not determine the bonus.

Multiplicative rarity has a different interpretation. Substituting the feature definitions into Eq. (3) gives

$$\frac{g_v r_v}{q_v} = \frac{\bar{c}}{|U|} \frac{1}{c_v} \sum_{u \in N[v] \cap U} \frac{1}{|N[u]|}. \tag{8}$$

The leading factor is shared by all candidates at a step, so RGR+P ranks vertices by total inverse-neighborhood weight of newly covered demand divided by candidate cost. Standard greedy values each newly covered vertex equally; RGR+P assigns more weight to demand vertices with fewer possible covering neighbors. Additive rarity instead places an average-rarity bonus alongside normalized coverage. It is not the same weighted-coverage calculation.

Table 5 summarizes all five winners. For the three non-seed winners, zeroing rarity changes training excess from -0.240% to $+0.349\%$, primary excess from $+0.522\%$ to $+0.334\%$, and BA excess from $+2.396\%$ to $+0.082\%$.

The rarity term helps training fitness but its contribution does not transfer to these held-out averages. Runs 1 and 2 reduce exactly to greedy when rarity is zeroed; run 4 retains its overlap penalty. Omitting pruning also worsens the five-winner averages, showing the importance of the fixed scaffold.

Table 5: Interventions on all five frozen EVO-F winners. Mean excess (%) is relative to pruned greedy on training (30 graphs), primary and BA (60 graphs each). No program is refit or reselected.

Intervention	Train	Primary	BA
Locked	-0.144	+0.313	+1.438
$r = 0$	+0.209	+0.200	+0.049
$o = 0$	-0.154	+0.141	+1.430
No pruning	+6.922	+7.558	+10.642

RGR+P, by contrast, improves mean cost on BA and both external weight families (Tables 2 and 3). Together, the algebra and interventions make the functional form of rarity weighting a plausible explanation for transfer differences. They analyze the selected heuristics' behavior without establishing a unique cause of the discovery policies' outcomes.

6.3 Illustrative Decision Trajectories

The following saved traces are the first qualifying primary improvement and BA failure in lexicographic (run, instance) order. They illustrate how scoring changes interact with construction and pruning.

A primary improvement. For primary instance 0014, run 1 uses f_1 from Eq. (6). At the third selection, vertices 15 and 35 both cover seven new vertices at cost two. Greedy selects 15 by ID; the higher average rarity makes EVO-F select 35. EVO-F finishes at cost 29, the certified optimum, versus greedy's 31. Vertex 35 is itself pruned: its effect is on the subsequent construction trajectory, rather than final membership. The strict improvement survives one of ten saved vertex relabelings; the other nine tie greedy, showing the example's sensitivity to ID-based tie-breaking.

A BA failure. For BA instance 0044, run 4 uses f_4. It agrees with greedy for four selections, then chooses vertex 30 instead of vertex 19. Their new-coverage-to-cost ratios are 1 and 2, respectively. The additive rarity term reverses this ranking even with the overlap penalty; without rarity, the two candidates retain greedy's ordering. Both trajectories later prune vertex 30. Final costs are 34 for EVO-F and 33 for greedy, while RGR+P reaches the certified optimum of 30. The first ranking reversal identifies a departure that propagates through later choices; it does not isolate the entire final loss to one decision. Complete traces and per-winner summaries remain in the repository.

7 Discussion and Limitations

The proposal-policy comparisons do not establish a reliable held-out improvement from parent conditioning or the additional prompt feedback. EVO-P already uses fitness to select its archive; EVO-F adds numerical summaries and case descriptors, after which parent histories can diverge. The study therefore compares complete search policies, rather than isolating the causal effect of numerical values in otherwise matched prompts.

The frozen-program evidence is more specific about behavior than discovery. Scoring changes often vanish before the objective is measured; when they survive, rarity's functional form changes how coverage is valued. This connects transfer to the representation, but leaves archive conditioning and the training corpus as competing explanations for which programs a policy selects.

The scope is limited by five runs per strategy, one shared training corpus and a restricted DSL. Matched proposal attempts yield unequal numbers of unique executable candidates. External evaluation covers 24 topologies under two weight regimes, with few certified optima; T1/T2 change both cost magnitudes and their relationship to degree. These constraints limit attribution and generalization. The reference-implementation timings also leave open what optimized learned heuristics and stronger large-scale classical controls would achieve.

8 Conclusion

The evaluated evolutionary policies do not establish a held-out advantage over independent generation or a clear incremental benefit from the prompt-feedback policy. Scoring diversity often produces trajectory diversity without changing the final solution or cost. The form of rarity weighting helps explain differences in frozen-program transfer, while CP-SAT remains the stronger control on the original small graphs. Broader representations and varied training corpora are needed to test whether these search policies yield more useful deployed heuristics.

9 Reproducibility

Source code, configurations, selected heuristics and numerical results are available in the accompanying repository. The README gives commands for reproducing tables and analyses from saved observations without LLM calls; `scripts/analyze_selected.py` derives the winner-level mechanism summaries. The original environment is Python 3.12.7 on Windows 11, NetworkX 3.7, OR-Tools 9.15.6755, NumPy 2.5.4 and Matplotlib 3.11.2.

Generative AI disclosure

Generative AI tools assisted literature search, LaTeX formatting and figures, drafting and polishing text, mathematical calculations and derivations, code editing, debugging, and experiment execution. Discussions with these tools helped refine ideas, methodology and hypotheses. The heuristic-generation model and its experimental role are documented separately. The author reviewed the AI-assisted work and takes responsibility for the final manuscript.

References

[1] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. https://doi.org/10.1038/s41586-023-06924-6.

[2] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32201–32223, 2024. https://proceedings.mlr.press/v235/liu24bs.html.

[3] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, volume 37, pages 43571–43608, 2024. https://doi.org/10.52202/079017-1381.

[4] Vašek Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979. https://doi.org/10.1287/moor.4.3.233.

[5] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131v1, 2025. https://arxiv.org/abs/2506.13131v1.

[6] Niki van Stein and Thomas Bäck. LLaMEA: A large language model evolutionary algorithm for automatically generating metaheuristics. arXiv:2405.20132v4, 2025. https://arxiv.org/abs/2405.20132v4.

[7] Edmund K. Burke, Michel Gendreau, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Özcan, and Rong Qu. Hyper-heuristics: a survey of the state of the art. *Journal of the Operational Research Society*, 64(12):1695–1724, 2013. https://doi.org/10.1057/jors.2013.71.

[8] Yiyuan Wang, Shaowei Cai, Jiejiang Chen, and Minghao Yin. A fast local search algorithm for minimum weight dominating set problem on massive graphs. In *Proceedings of IJCAI*, pages 1514–1522, 2018. https://doi.org/10.24963/ijcai.2018/210.

[9] Lukas Dijkstra, Andrei Gagarin, and Vadim Zverovich. Weighted domination models and randomized heuristics. *Utilitas Mathematica*, 123:61–86, 2025. https://doi.org/10.61091/um123-05.

[10] Kevin Sim, Quentin Renau, and Emma Hart. Beyond the hype: Benchmarking LLM-evolved heuristics for bin packing. In *Applications of Evolutionary Computation*, volume 15613 of *Lecture Notes in Computer Science*, pages 386–402. Springer, 2025. https://doi.org/10.1007/978-3-031-90065-5_24.

[11] NetworkX Developers. `barabasi_albert_graph`: API documentation. https://networkx.org/documentation/stable/reference/generated/networkx.generators.random_graphs.barabasi_albert_graph.html. Accessed 5 November 2026. Experiment uses NetworkX 3.7.

[12] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall, 1993. https://doi.org/10.1007/978-1-4899-4541-9.

[13] Google OR-Tools. CP-SAT solver. https://developers.google.com/optimization/cp/cp_solver. Accessed 5 November 2026. Experiment uses OR-Tools 9.15.6755.

[14] Raka Jovanović, Milan Tuba, and Dana Simian. Ant colony optimization applied to minimum weight dominating set problem. In *Proceedings of the 12th WSEAS International Conference on Automatic Control, Modelling and Simulation*, pages 322–326, 2010. Original T1/T2 instances: https://mail.ipb.ac.rs/~rakaj/home/BenchmarkMWDSP.htm.